



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table><tr><th>Student 1</th><th>Student 2</th></tr><tr><td>No. 1, 3, 5</td><td>No 2, 4, 6</td></tr></table> Name 1: ANIS SAFIYYA BINTI JANAI Metric No: A23CS0049 Name 2: YASMIN BATRISYIA BINTI ZAHIRUDDIN Metric No: A23CS0201	Student 1	Student 2	No. 1, 3, 5	No 2, 4, 6
Student 1	Student 2				
No. 1, 3, 5	No 2, 4, 6				
SUBMISSION DATE:	23/05/2024				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

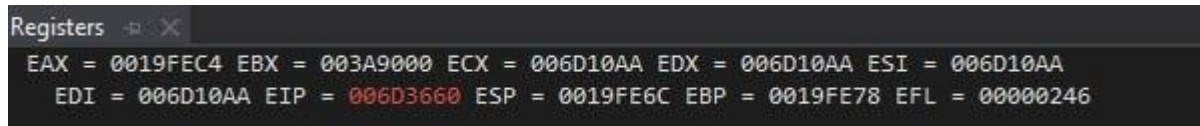
a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 (9d) into register BX
LINE3	AX = 0009h BX = 0001h	Changes the contents of AX and BX
LINE4	AX = 0009h var1 = 0009h	Move the value of register AX (9d) into var1
LINE5	BX = 0001h var2 = 0001h	Move the value of register AX (9d) into var1

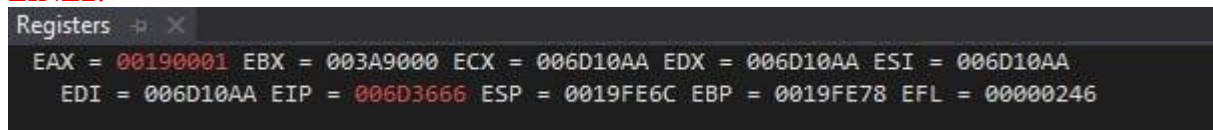
b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



```
Registers
EAX = 0019FEC4 EBX = 003A9000 ECX = 006D10AA EDX = 006D10AA ESI = 006D10AA
EDI = 006D10AA EIP = 006D3660 ESP = 0019FE6C EBP = 0019FE78 EFL = 00000246
```

LINE2:



```
Registers
EAX = 00190001 EBX = 003A9000 ECX = 006D10AA EDX = 006D10AA ESI = 006D10AA
EDI = 006D10AA EIP = 006D3666 ESP = 0019FE6C EBP = 0019FE78 EFL = 00000246
```

LINE3:

```
Registers  + X
EAX = 00190001 EBX = 003A0009 ECX = 006D10AA EDX = 006D10AA ESI = 006D10AA
EDI = 006D10AA EIP = 006D366D ESP = 0019FE6C EBP = 0019FE78 EFL = 00000246
```

LINE4:

```
Registers  + X
EAX = 00190009 EBX = 003A0001 ECX = 006D10AA EDX = 006D10AA ESI = 006D10AA
EDI = 006D10AA EIP = 006D366F ESP = 0019FE6C EBP = 0019FE78 EFL = 00000246
```

LINE5:

```
Registers  + X
EAX = 00190009 EBX = 003A0001 ECX = 006D10AA EDX = 006D10AA ESI = 006D10AA
EDI = 006D10AA EIP = 006D3675 ESP = 0019FE6C EBP = 0019FE78 EFL = 00000246
```

Call DumpRegs:

```
Registers  + X
EAX = 00190009 EBX = 003A0001 ECX = 006D10AA EDX = 006D10AA ESI = 006D10AA
EDI = 006D10AA EIP = 006D367C ESP = 0019FE6C EBP = 0019FE78 EFL = 00000246
```

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval      ; LINE3
    sub ebx,Zval      ; LINE4
    add eax,ebx       ; LINE5
    inc eax           ; LINE6
    mov Rval,eax      ; LINE7
    exit
main endp
end main
```

Answer Q2

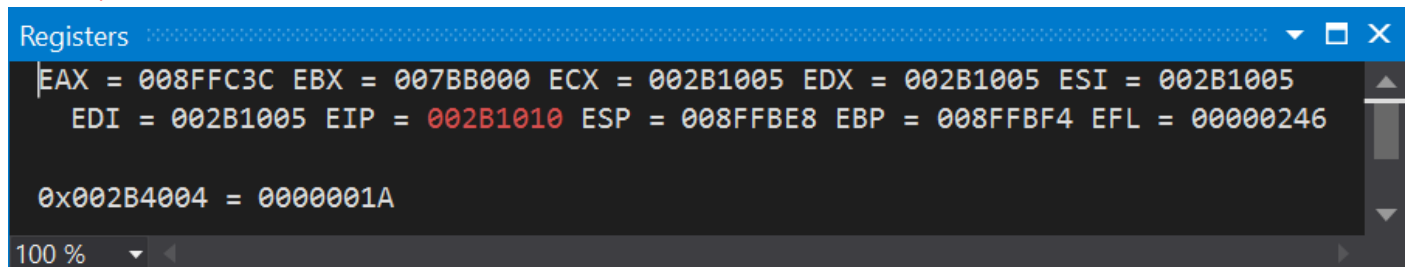
a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFE6h	Negate the value Xval (26d) to (-26d)
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX = FFFFFFFF6h Zval = 00000028h	Subtract the value of EBX (30d) with Zval (40d) = -10d and stored it in EBX
LINE5	EAX = FFFFFFFDCh EBX = FFFFFFFF6h	Add the value of EAX (-26d) and EBX (-10d) = -36d and stored it EAX
LINE6	EAX = FFFFFFFDDh	Add 1 to EAX (-36d) = -35d into EAX
LINE7	EAX = FFFFFFFDDh Rval = FFFFFFFDDh	Move the EAX value (-35d) into Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

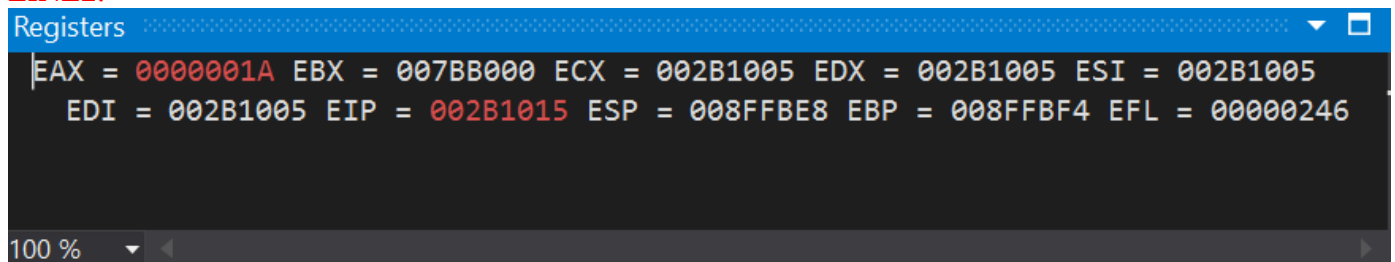
LINE1:



```
Registers
EAX = 008FFC3C EBX = 007BB000 ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B1010 ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000246

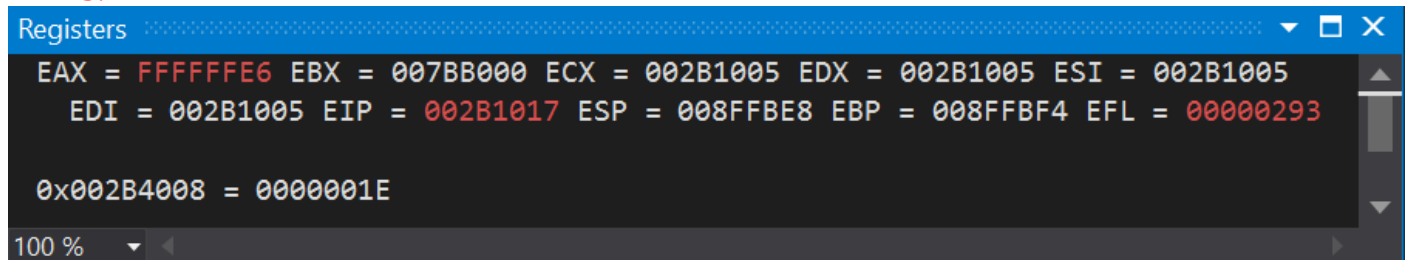
0x002B4004 = 0000001A
100 %
```

LINE2:



```
Registers
EAX = 0000001A EBX = 007BB000 ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B1015 ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000246
100 %
```

LINE3:



```
Registers
EAX = FFFFFFFE6 EBX = 007BB000 ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B1017 ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000293

0x002B4008 = 0000001E
100 %
```

LINE4:

```
Registers
EAX = FFFFFFFE6 EBX = 0000001E ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B101D ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000293

0x002B400C = 00000028

100 %
```

LINE5:

```
Registers
EAX = FFFFFFFE6 EBX = FFFFFFFF6 ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B1023 ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000287

100 %
```

LINE6:

```
Registers
EAX = FFFFFFFDC EBX = FFFFFFFF6 ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B1025 ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000283

100 %
```

LINE7:

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B1026 ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000287

0x002B4000 = 00000000

100 %
```

EXIT:

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 002B1005 EDX = 002B1005 ESI = 002B1005
EDI = 002B1005 EIP = 002B102B ESP = 008FFBE8 EBP = 008FFBF4 EFL = 00000287

100 %
```

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

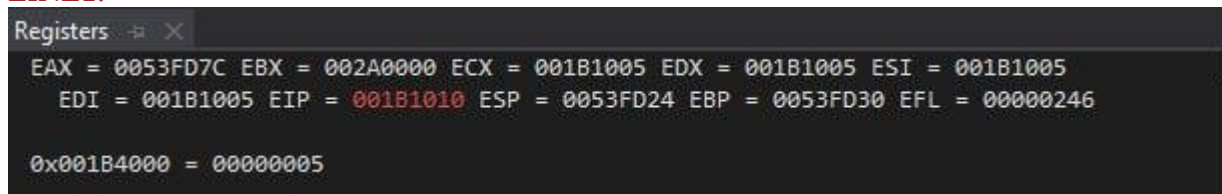
a) Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	Multiplies the value of var2 (10d) by EAX (5d) = 50d and store it in EDX:EAX
LINE3	EAX = 00000046h var3 = 00000014h	Add the value of var3 (20d) into EAX (50d) = 70d and store it in EAX
LINE4	EAX = 00000045h var4 = 00000000h	Subtract 1 from register EAX (70d)

b) Paste here screenshot of all registers' content after each LINE is executed:

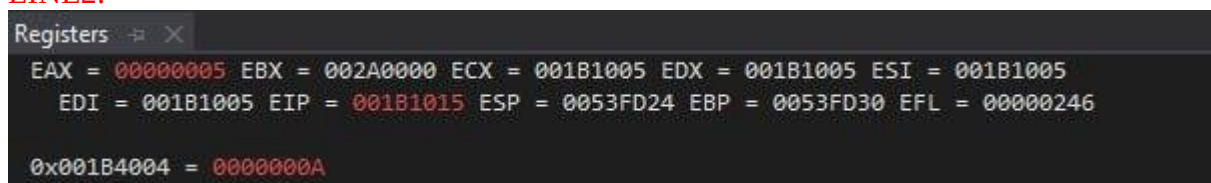
LINE1:



```
Registers  ⇅  ✕
EAX = 0053FD7C EBX = 002A0000 ECX = 001B1005 EDX = 001B1005 ESI = 001B1005
EDI = 001B1005 EIP = 001B1010 ESP = 0053FD24 EBP = 0053FD30 EFL = 00000246

0x001B4000 = 00000005
```

LINE2:



```
Registers  ⇅  ✕
EAX = 00000005 EBX = 002A0000 ECX = 001B1005 EDX = 001B1005 ESI = 001B1005
EDI = 001B1005 EIP = 001B1015 ESP = 0053FD24 EBP = 0053FD30 EFL = 00000246

0x001B4004 = 0000000A
```

LINE3:

```
Registers  ➤ ✕  
EAX = 00000032 EBX = 002A0000 ECX = 001B1005 EDX = 00000000 ESI = 001B1005  
EDI = 001B1005 EIP = 001B1018 ESP = 0053FD24 EBP = 0053FD30 EFL = 00000202  
  
0x001B4008 = 00000014
```

LINE4:

```
Registers  ➤ ✕  
EAX = 00000046 EBX = 002A0000 ECX = 001B1005 EDX = 00000000 ESI = 001B1005  
EDI = 001B1005 EIP = 001B1021 ESP = 0053FD24 EBP = 0053FD30 EFL = 00000202
```

EXIT:

```
Registers  ➤ ✕  
EAX = 00000045 EBX = 002A0000 ECX = 001B1005 EDX = 00000000 ESI = 001B1005  
EDI = 001B1005 EIP = 001B1022 ESP = 0053FD24 EBP = 0053FD30 EFL = 00000202
```

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

Answer Q4

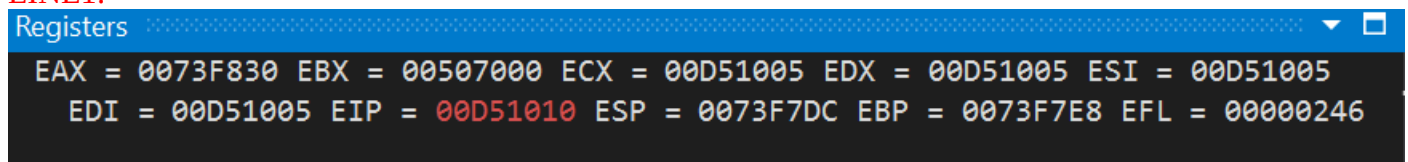
a) Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	Move the value 5d into register BX
LINE3	AX = 00C8h BX = 0005h	Multiplies the value of BX (5d) by AX (40d) = 200d and store it in AX
LINE4	BX = 000Ah var2 = 000Ah	Move the value of var2 (10d) into register BX
LINE5	BX = 0007h	Move value var2 (10d) into register BX
LINE6	AX = 001Ch BX = 0007h DX = 0004h	Divide AX (200d) by BX (7d) = AX (28d) and DX (4d)
LINE7	AX = 001Ch var4 = 001Ch	Move the value of register AX () into var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:

Registers

EAX = 00730028 EBX = 00507000 ECX = 00D51005 EDX = 00D51005 ESI = 00D51005
EDI = 00D51005 EIP = 00D51016 ESP = 0073F7DC EBP = 0073F7E8 EFL = 00000246

LINE3:

Registers

EAX = 00730028 EBX = 00500005 ECX = 00D51005 EDX = 00D51005 ESI = 00D51005
EDI = 00D51005 EIP = 00D5101A ESP = 0073F7DC EBP = 0073F7E8 EFL = 00000246

LINE4:

Registers

EAX = 007300C8 EBX = 00500005 ECX = 00D51005 EDX = 00D50000 ESI = 00D51005
EDI = 00D51005 EIP = 00D5101D ESP = 0073F7DC EBP = 0073F7E8 EFL = 00000202

LINE5:

Registers

EAX = 007300C8 EBX = 0050000A ECX = 00D51005 EDX = 00D50000 ESI = 00D51005
EDI = 00D51005 EIP = 00D51024 ESP = 0073F7DC EBP = 0073F7E8 EFL = 00000202

LINE6:

Registers

EAX = 007300C8 EBX = 00500007 ECX = 00D51005 EDX = 00D50000 ESI = 00D51005
EDI = 00D51005 EIP = 00D51028 ESP = 0073F7DC EBP = 0073F7E8 EFL = 00000202

LINE7:

Registers

EAX = 0073001C EBX = 00500007 ECX = 00D51005 EDX = 00D50004 ESI = 00D51005
EDI = 00D51005 EIP = 00D5102B ESP = 0073F7DC EBP = 0073F7E8 EFL = 00000202

EXIT:

Registers

EAX = 0073001C EBX = 00500007 ECX = 00D51005 EDX = 00D50004 ESI = 00D51005
EDI = 00D51005 EIP = 00D51031 ESP = 0073F7DC EBP = 0073F7E8 EFL = 00000202

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
 - b. The divisor is specified as the operand of the DIV instruction.
 - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
 - d. The remainder is stored in DX.
 - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
 - f. If the divisor is 0, a division error occurs.
 - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX, 0           ; Clear DX
MOV AX, 1000h        ; Load 1000h into AX
MOV CX, 25h          ; Load 25h into CX
MUL CX               ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

a) Screenshot of full program (.asm) :

```
1  INCLUDE Irvine32.inc
2
3  .code
4  main PROC
5      MOV DX, 0; Clear DX
6      MOV AX, 1000h; Load 1000h into AX
7      MOV CX, 25h; Load 25h into CX
8      MUL CX; Multiply AX by CX, storing the result in DX : AX
9      call DumpRegs
10     exit
11
12     main ENDP
13     END main
```

b) Paste here the screenshot of the final registers' content (DumpReg):

```
Registers  [X]
EAX = 00EF5000 EBX = 00DD2000 ECX = 00B20025 EDX = 00B20002 ESI = 00B210AA
EDI = 00B210AA EIP = 00B2366F ESP = 00EFFCA0 EBP = 00EFFCAC EFL = 00000A07
```

Q6. Given the following instructions as is Code Snippet 2.

a) Write a full program to execute the Code Snippet 2.

b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

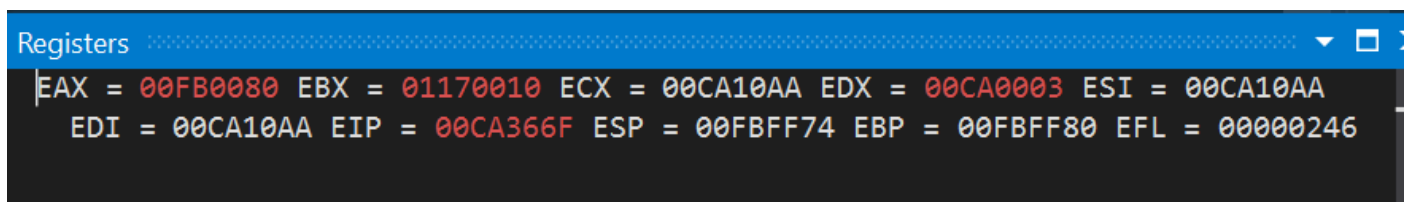
```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

a) Screenshot of full program (.asm) :

```
1  include Irvine32.inc
2
3  .code
4  main proc
5  MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
6  MOV AX, 803h; Load the dividend(8003h) into AX
7  MOV BX, 10h; Load the divisor(10h) into BX
8  DIV BX; Divide DX : AX by BX, whereby AX = quotient & DX = remainder
9  call DumpRegs
10 exit
11 main endp
12 end main
13
```

b) Paste here the screenshot of the final registers' content (DumpReg):



The screenshot shows the 'Registers' window with the following values:

Register	Value
EAX	00FB0080
EBX	01170010
ECX	00CA10AA
EDX	00CA0003
ESI	00CA10AA
EDI	00CA10AA
EIP	00CA366F
ESP	00FBFF74
EBP	00FBFF80
EFL	00000246

YouTube video link:

<https://youtu.be/tK8Ezs3kYw0?si=TV6IeXXzvWZWYwrx>